

TRUSTCHAIN PRO

Blockchain-Based Secure Platform for Identity, Access Control & Digital Asset Management

SIH26125 • BLOCKCHAIN & CYBERSECURITY • SOFTWARE

PROJECT TEAM

TECH FORCE

Department of Computer Engineering
Zeal College of Engineering and Research

IDENTITY	ACCESS	ASSETS	GOVERNANCE
DID + ECDSA P-256	RBAC + ABAC	Enterprise NFTs	Chaincode Rules

PROJECT SNAPSHOT

One platform. Four security layers. One auditable asset lifecycle.

THE CHALLENGE Identity, authorization, asset ownership and audit records are often fragmented across centralized systems—making privileged actions harder to verify independently.	THE CORE IDEA Bind cryptographic identity to role and live context, then govern every sensitive asset operation through explicit policy and a tamper-evident ledger.
THE DIFFERENTIATOR Enterprise NFTs provide unique, traceable digital ownership while chaincode rules govern who may mint, allocate or transfer an asset.	THE DEPLOYMENT MODEL MERN + permissioned infrastructure + REST/WebSockets keeps the solution practical for controlled intranets without public-chain gas or Proof-of-Work.

SECURITY DECISION FLOW



Problem Statement: SIH26125 • Organization: Bharat Electronics Limited

ACKNOWLEDGEMENT

We express our sincere gratitude to our faculty members, project mentors and institution for their guidance and support during the development of TrustChain Pro.

We also acknowledge the Smart India Hackathon platform and the organization associated with Problem Statement SIH26125 for providing a practical problem context around secure identity, access control and digital asset management.

We are grateful to the open-source communities and standards organizations whose documentation and specifications helped shape the proposed architecture, including W3C, NIST, Node.js, MongoDB and related cryptography and security references.

• **TECH FORCE** • **ZEAL COLLEGE OF ENGINEERING AND RESEARCH**

EXECUTIVE SUMMARY

TrustChain Pro is a proposed secure digital platform for managing identity, access permissions, and ownership of digital assets in defense, government, and other high-assurance environments. The project addresses a central problem: conventional centralized systems can make identity, authorization, ownership, and audit trails difficult to verify independently when systems are compromised or when privileged access is abused.

TrustChain Pro combines passwordless Decentralized Identifiers (DIDs), ECDSA P-256 signatures, role-based access control (RBAC), attribute-based access control (ABAC), Enterprise NFTs, smart-contract/chaincode governance, and a permissioned tamper-evident ledger. The model requires both the user's assigned role and the current security context to satisfy policy before a protected operation is accepted.

The four foundational RBAC roles are Admin, Manager, Auditor, and User. ABAC adds dynamic conditions such as secure IP/network context, time, clearance, asset sensitivity, and other policy attributes. Enterprise NFTs represent digital assets as unique ledger objects with traceable ownership, while chaincode-style governance rules control who may mint, allocate, or transfer them.

The proposed implementation uses a MERN-based application layer with Web Crypto API for client-side cryptographic operations, Node.js/Express for policy and API services, Socket.io/WebSockets for real-time synchronization, and MongoDB-backed hash chaining for the prototype ledger. A permissioned validator topology avoids public blockchain gas costs and Proof-of-Work mining.

TABLE OF CONTENTS

1. Introduction

- 1.1 Introduction
- 1.2 Aim of the Project
- 1.3 Problem Statement
- 1.4 Objectives
- 1.5 Scope of the Project

2. Background and Existing System

- 2.1 Background
- 2.2 Existing System
- 2.3 Limitations of Existing Approaches
- 2.4 Need for the Proposed System

3. Proposed Solution

- 3.1 TrustChain Pro Overview
- 3.2 Core Solution Pillars
- 3.3 Identity Layer
- 3.4 Hybrid Access Control
- 3.5 Enterprise NFT Asset Ownership
- 3.6 Smart Contracts / Chaincode Governance
- 3.7 Tamper-Evident Ledger and Auditability

4. Project Implementation

- 4.1 User Roles
- 4.2 Registration and Identity Provisioning
- 4.3 Authentication Flow
- 4.4 Authorization Flow
- 4.5 Digital Asset Lifecycle
- 4.6 Asset Transfer and Governance
- 4.7 Revocation and Key Recovery
- 4.8 Audit and Forensic Monitoring

5. Technical Architecture

- 5.1 Architecture Overview
- 5.2 Frontend Layer
- 5.3 Backend Layer
- 5.4 Permissioned Ledger Layer
- 5.5 Database and Hash Chaining
- 5.6 REST API and WebSocket Integration
- 5.7 Consensus and Validator Nodes

- 5.8 Cryptographic Design

6. Major Technical Requirements

- 6.1 Hardware Requirements
- 6.2 Software Requirements
- 6.3 Technology Stack
- 6.4 Libraries and APIs
- 6.5 Data Model
- 6.6 Security Requirements

7. Feasibility and Viability

- 7.1 Technical Feasibility
- 7.2 Economic Feasibility
- 7.3 Operational Scalability
- 7.4 Legal and Regulatory Considerations
- 7.5 Viability and Adoption
- 7.6 Potential Deployment

8. Challenges and Limitations

- 8.1 Key Management
- 8.2 Performance and Latency
- 8.3 Insider and Privilege Risks
- 8.4 Node Availability
- 8.5 Integration Challenges
- 8.6 Current Prototype Limitations

9. Impact and Benefits

- 9.1 Security Impact
- 9.2 Operational Benefits
- 9.3 Asset Governance Benefits
- 9.4 Audit and Accountability
- 9.5 Government and Defense Use Cases

10. Future Enhancements

- 10.1 Advanced Key Recovery
- 10.2 Hardware-Backed Identity
- 10.3 Advanced Policy Engine
- 10.4 Cross-Department Federation
- 10.5 Advanced SIEM Integration
- 10.6 Mobile and Offline Workflows

11. Conclusion

12. References

1. INTRODUCTION

1.1 Introduction

Digital identity and access management are foundational components of modern government and defense information systems. As organizations move from paper records and isolated applications to interconnected digital platforms, the ability to verify who is requesting access, why access is being requested, whether the request is safe in the current context, and what happened afterward becomes increasingly important.

TrustChain Pro proposes a security-oriented platform that treats identity, access, asset ownership, and auditability as connected components rather than separate functions. A cryptographically verifiable identity is used to authenticate a user; RBAC establishes the user's organizational role; ABAC evaluates dynamic context; governance rules determine whether an asset operation is allowed; and the resulting event is recorded in a tamper-evident ledger.

1.2 Aim of the Project

The aim is to design a practical, deployable, and auditable platform for secure identity, access control, and digital asset management in controlled organizational environments. The solution is intended to improve protection against unauthorized access, privilege misuse, weak ownership tracking, and silent modification of critical records.

- Provide passwordless cryptographic identity using DIDs and ECDSA P-256.
- Combine static organizational roles with dynamic context-aware access policies.
- Represent important digital assets as unique, traceable Enterprise NFTs.
- Automate asset governance through smart-contract/chaincode rules.
- Maintain tamper-evident transaction and ownership history.
- Support integration with existing intranets through REST APIs and WebSockets.

1.3 Problem Statement

Problem Statement SIH26125, "Blockchain-Based Secure Platform for Identity, Access Control, and Digital Asset Management", calls for a secure platform capable of managing identity, access permissions, and digital assets with stronger traceability and governance. The project focuses on reducing dependence on purely centralized authorization decisions and improving the evidence available when sensitive operations occur.

1.4 Objectives

1. Create a passwordless identity layer based on decentralized identifiers and digital signatures.
2. Implement four foundational RBAC roles: Admin, Manager, Auditor, and User.
3. Add ABAC conditions so that a correct role alone is not sufficient when the current context is unsafe.
4. Tokenize selected enterprise digital assets as unique, traceable NFT-like ledger objects.
5. Enforce mint, allocation, and transfer policies through programmable governance rules.
6. Record important events using hash-linked, tamper-evident transaction structures.
7. Provide real-time synchronization and audit visibility for authorized stakeholders.

1.5 Scope of the Project

The project covers the application-layer identity, access, asset governance, ledger, audit, and integration mechanisms required for a proof-of-concept platform. It does not claim to replace national identity infrastructure, classified-information accreditation, hardware security modules, or a formally certified blockchain implementation. Production deployment would require security assessment, infrastructure hardening, key-management procedures, and organization-specific policy approval.

2. BACKGROUND AND EXISTING SYSTEM

2.1 Background

Traditional enterprise applications often maintain identity records, access control lists, asset databases, and audit logs in separate systems. Centralization simplifies administration but also creates attractive targets for credential theft, privilege abuse, database manipulation, and single-point failures. In high-assurance environments, an audit record is useful only when its integrity can be demonstrated.

Blockchain-inspired ledger mechanisms provide a way to make records tamper-evident through cryptographic linking. Permissioned architectures are especially relevant to government and enterprise settings because participating nodes can be controlled, identities can be known, and transaction policies can be defined by the organization.

2.2 Existing System

- Username/password or centrally managed credentials are commonly used for application access.
- Authorization is often primarily role-based and may not consider current network or device context.
- Digital asset ownership can remain in conventional database records without cryptographic token identity.
- Audit logs may be stored centrally and can become difficult to trust after a privileged compromise.
- Asset transfers may depend on manual approval workflows and are not always represented as programmable rules.

2.3 Limitations of Existing Approaches

Area	Common Limitation	TrustChain Pro Direction
Identity	Credential theft and password reuse	Passwordless DID + ECDSA signing
Access	Static permissions may ignore context	RBAC + dynamic ABAC
Ownership	Database record may not provide unique asset identity	Enterprise NFT-style asset token
Governance	Manual or application-specific approval logic	Chaincode/smart-contract rules
Audit	Central logs can be modified or deleted	Hash-linked tamper-evident records
Integration	Security solutions may be isolated	REST APIs + WebSockets

2.4 Need for the Proposed System

A secure system must connect identity to authorization, authorization to asset operations, and asset operations to evidence. TrustChain Pro is designed around this chain of trust so that an operation is not treated as an isolated database update. The user must be authenticated, the role must be appropriate, the current context must satisfy policy, the governance rule must permit the action, and the resulting event must be auditable.

3. PROPOSED SOLUTION

3.1 TrustChain Pro Overview

TrustChain Pro is a permissioned, blockchain-inspired enterprise security platform. Its architecture combines conventional web technologies with cryptographic identity and ledger mechanisms. The purpose is not to introduce cryptocurrency or public-chain speculation; it is to use verifiable cryptographic records and controlled governance for enterprise security.

3.2 Core Solution Pillars

Pillar	Purpose	Key Technology
Identity	Prove who is making a request	DID, ECDSA P-256
Access	Decide whether the request is permitted	RBAC + ABAC
Asset	Represent and track digital ownership	Enterprise NFTs
Governance	Control mint/allocate/transfer actions	Smart Contracts / Chaincode
Ledger	Preserve transaction evidence	SHA-256 hash chaining
Integration	Connect to existing systems	REST APIs, WebSockets

3.3 Identity Layer

Each participating user is associated with a decentralized identifier. The user's private key remains under the user's control in the intended model, while the public verification material is associated with the identity record. Requests are signed using ECDSA P-256. The server verifies the signature before evaluating authorization policies.

- Passwordless authentication reduces dependence on reusable passwords.
- ECDSA P-256 provides a widely supported elliptic-curve signature mechanism.
- Client-side Web Crypto API can perform key and signature operations in the browser.
- Identity history can remain associated with a stable organizational record even when keys are rotated under an approved recovery procedure.

3.4 Hybrid Access Control

TrustChain Pro uses RBAC as the foundational permission layer and ABAC as a contextual gatekeeper. RBAC answers “what is this user allowed to do in principle?” while ABAC asks “is this request safe under the current conditions?” A request should satisfy both layers before sensitive operations proceed.

RBAC Role	Typical Permissions
Admin	Manage identities, roles, governance policies and system configuration
Manager	Approve or initiate permitted operational asset actions
Auditor	Read audit history, verify records and investigate events
User	Perform authorized day-to-day operations within assigned scope

Dynamic attributes can include secure IP/network, time window, clearance level, asset sensitivity, device posture, request type, and other organization-defined conditions. The exact policy values should be configured by the deploying organization.

3.5 Enterprise NFT Asset Ownership

The Enterprise NFT concept represents a digital asset as a unique, cryptographically identifiable ledger object. Examples include sensitive documents, licenses, credentials, certificates, software entitlements, equipment records, or other organizational digital assets. The token records the asset identity and ownership relationship without requiring a public cryptocurrency network.

- Unique asset identity prevents ambiguous ownership records.
- Ownership and transfer history remain traceable.
- Asset operations can be linked to the authenticated organizational identity.
- Tokens are governed by enterprise policy rather than open-market trading rules.

3.6 Smart Contracts / Chaincode Governance

Governance logic defines the rules under which an asset can be created, allocated, transferred, revoked, or retired. In a permissioned deployment, this logic can be implemented as chaincode or smart-contract-like services. The important principle is that asset state changes are policy-controlled rather than arbitrary database edits.

3.7 Tamper-Evident Ledger and Auditability

Each important event can be represented as a record containing transaction metadata, actor identity reference, action, asset reference, timestamp, and cryptographic linkage to the previous record. SHA-256 hashing provides the integrity mechanism for the hash chain. A mismatch indicates that a stored record or its linkage should be investigated.

4. PROJECT IMPLEMENTATION

4.1 User Roles

The project explicitly defines four roles so that identity and organizational responsibility remain understandable to users and administrators.

Role	Primary Responsibility	Example Actions
Admin	Platform and policy administration	Create users, assign roles, manage policies, emergency controls
Manager	Operational approval and asset management	Approve permitted allocations/transfers
Auditor	Independent verification and oversight	Inspect logs, verify hashes, generate audit evidence
User	Routine authorized usage	Read or operate assets allowed by policy

4.2 Registration and Identity Provisioning

- Administrator initiates or approves creation of a user identity.
- Cryptographic key material is generated using the approved client-side mechanism.
- A DID and public verification material are associated with the organizational identity.
- The administrator assigns one of the supported RBAC roles and required attributes.
- The user receives the approved identity context without requiring a conventional reusable password.

4.3 Authentication Flow

- User selects the protected operation.
- Client creates a request payload containing the required operation context.
- Private key signs the payload using ECDSA P-256.
- Backend verifies the signature against the registered public key.
- Only a successfully authenticated request proceeds to authorization.

4.4 Authorization Flow

- RBAC checks the user's role and baseline permission.
- ABAC evaluates contextual attributes such as IP/network, time, clearance, asset sensitivity, and policy state.
- Governance rules verify whether the requested asset operation is allowed.
- If all required checks pass, the operation is committed and recorded.
- If a check fails, the operation is denied and the event can be logged for audit.

4.5 Digital Asset Lifecycle

Stage	Description
Create	Authorized party requests creation of a digital asset record.
Mint	Governance rules create a unique Enterprise NFT representation.
Allocate	Asset is assigned to an approved identity or department.
Use	Authorized user performs permitted read/use operations.

Transfer	Governed transfer changes the recorded ownership relationship.
Revoke	Asset or permission is invalidated under policy.
Archive	Final state and audit history are retained for the required period.

4.6 Asset Transfer and Governance

A sensitive transfer should not be treated as a normal database update. The request is authenticated, evaluated against RBAC and ABAC, checked against the asset's governance policy, and then recorded. For particularly critical assets, the deployment can require multi-party approval before the transfer is finalized.

4.7 Revocation and Key Recovery

A live revocation registry can be used to mark a compromised or deactivated key as invalid without rewriting historical records. A production key-recovery process should use organizational verification and controlled re-issuance. The proposed model can preserve a linkage between the historical identity record and the newly issued key.

4.8 Audit and Forensic Monitoring

The audit layer provides authorized auditors with searchable event history. Hash verification can identify records whose content or chain linkage has changed. A SIEM-style dashboard can surface suspicious patterns such as repeated denied access, unusual transfers, revoked-key usage, or hash mismatches.

5. TECHNICAL ARCHITECTURE

5.1 Architecture Overview

The architecture is divided into presentation, application, governance, ledger, and integration layers. The design deliberately uses established web technologies so that implementation and maintenance can be performed by standard software teams rather than requiring a specialized public-blockchain development organization.

Layer	Components	Responsibilities
Client	React.js, Web Crypto API	UI, key operations, request signing
Application	Node.js, Express.js	APIs, authentication, authorization, policy evaluation
Real-time	Socket.io / WebSockets	Event synchronization and status updates
Governance	RBAC + ABAC + chaincode rules	Permission and asset-state decisions
Ledger	Permissioned nodes, hash-linked records	Transaction integrity and shared state
Database	MongoDB	Structured application data and ledger persistence
Audit	Verification/SIEM dashboard	Integrity checks, investigation and reporting

5.2 Frontend Layer

React.js provides the web interface for identity management, asset dashboards, approvals, audit views, and administrative functions. The Web Crypto API can perform browser-side cryptographic operations so that private key material does not need to be sent to the server as part of routine signing.

5.3 Backend Layer

Node.js with Express.js provides REST endpoints for identity, access, asset, governance, and audit operations. The backend validates signatures, loads role and attribute information, evaluates policies, and coordinates ledger commits.

5.4 Permissioned Ledger Layer

The proposed deployment uses a controlled validator network. Unlike a public cryptocurrency network, validators are known organizational nodes and the network does not depend on public mining. This makes the architecture appropriate for controlled government or enterprise environments where membership and governance must be managed.

5.5 Database and Hash Chaining

MongoDB can store application records and the prototype's ledger structures. A hash-linked record can contain the current transaction data and the hash of the previous record. A simplified integrity relationship is:

$$\text{Current Hash} = \text{SHA-256}(\text{Current Record Data} + \text{Previous Hash})$$

This structure makes silent modification detectable because changing an earlier record changes its hash and breaks the subsequent chain.

5.6 REST API and WebSocket Integration

REST APIs allow existing intranet applications to request identity, authorization, asset, and audit services without replacing their complete application stack. WebSockets provide event-driven updates for approval status, asset changes, validator synchronization, and security alerts.

5.7 Consensus and Validator Nodes

The prototype is designed around a small controlled validator topology, with a target of three nodes for a minimal deployment. A production implementation would select and formally specify the consensus mechanism based on the organization's reliability, fault tolerance, and security requirements.

5.8 Cryptographic Design

Mechanism	Purpose
DID	Decentralized identifier for an organizational identity
ECDSA P-256	Digital signatures for passwordless request authentication
SHA-256	Hashing and tamper-evident record linkage
Salted/derived references	Reduce exposure of raw personal identifiers in audit structures
Key rotation	Controlled response to compromised or lost credentials

6. MAJOR TECHNICAL REQUIREMENTS

6.1 Hardware Requirements

- Development workstation with modern 64-bit CPU, 8 GB or more RAM recommended, and reliable network connectivity.
- Server or virtual-machine resources appropriate for the expected number of users and validator nodes.
- Secure storage and backup facilities for approved key-management and recovery procedures in production.

6.2 Software Requirements

- Windows/Linux development environment as supported by the selected toolchain.
- Node.js and npm.
- React.js toolchain.
- MongoDB / MongoDB-compatible deployment.
- Modern web browser supporting Web Crypto API.
- Version control and standard development tooling.

6.3 Technology Stack

Technology	Use in TrustChain Pro
React.js	Frontend interface
Node.js	Backend runtime
Express.js	REST API layer
MongoDB	Application and prototype ledger persistence
Socket.io	Real-time synchronization
Web Crypto API	ECDSA and SHA-256 operations
Permissioned blockchain / ledger	Controlled validator-based transaction integrity
Chaincode / governance service	Asset lifecycle rules

6.4 Libraries and APIs

The implementation can use standard Node.js cryptography facilities, browser Web Crypto APIs, Express middleware, Socket.io, MongoDB drivers or Mongoose, and standard JSON-based APIs. Dependency selection should be pinned and regularly updated as part of production security maintenance.

6.5 Data Model

Entity	Important Fields
Identity	DID, public key, role, attributes, status, key version
User	Organization ID, role, department, clearance reference
Asset	Asset ID, token ID, owner, sensitivity, status, metadata hash
Policy	Role rules, ABAC conditions, permitted actions
Transaction	Actor, action, asset, timestamp, previous hash, current hash
Approval	Requester, approver(s), decision, timestamp, evidence
Revocation	Identity/key ID, reason, timestamp, status

6.6 Security Requirements

- Private keys must not be transmitted as ordinary API payloads.
- Sensitive endpoints must validate cryptographic identity before authorization.
- Authorization policies must be deny-by-default for protected operations.
- Critical transfers should support multi-party approval where required.
- Audit records must be protected from unauthorized modification.
- Access to administrative functions must be logged and reviewed.

7. FEASIBILITY AND VIABILITY

7.1 Technical Feasibility

The proposed system is technically feasible because it relies primarily on mature web technologies and well-established cryptographic primitives. MERN technologies provide the application foundation, while the permissioned ledger layer can be implemented without requiring public-chain cryptocurrency infrastructure.

- Mature MERN stack reduces dependence on experimental Web3 languages.
- Standard government IT teams can maintain the web application and policy layer.
- REST APIs and WebSockets enable integration with legacy intranet applications.
- Client-side ECDSA signing reduces the need to transmit private key material.

7.2 Economic Feasibility

- Private permissioned architecture does not require public blockchain gas fees.
- Node.js and MongoDB are available as open-source technologies.
- No Proof-of-Work mining infrastructure is required.
- Existing enterprise infrastructure can be reused where appropriate.

7.3 Operational Scalability

- Small controlled validator topology can support an initial deployment.
- Additional nodes/departments can be introduced as organizational requirements grow.
- WebSocket event delivery supports responsive operational dashboards.
- Client-side cryptographic operations can offload part of the processing burden.

7.4 Legal and Regulatory Considerations

The architecture is designed with privacy and audit requirements in mind, but implementation does not by itself establish legal compliance. Production deployment should be reviewed by the organization's legal, privacy, cybersecurity, and accreditation teams.

- Hash-based references can reduce the need to place raw personal identifiers in the ledger.
- Policy-based access control supports least-privilege and Zero-Trust design principles.
- Tamper-evident audit records can strengthen evidence handling and accountability.
- Retention, deletion, access, and disclosure policies must be configured to the applicable legal requirements.

7.5 Viability and Adoption

Viability depends on operational value, integration effort, governance readiness, and the ability to maintain the system over time. TrustChain Pro is designed to fit existing organizational workflows rather than requiring a complete replacement of existing intranet systems.

Viability Factor	TrustChain Pro Approach
Mission Value	Secure identity, access and critical digital asset ownership
Adoption	Existing web technologies and API integration
Scalability	Modular permissioned validator architecture
Auditability	Hash-linked transaction history
Sustainability	Open-source core technologies and no public-chain gas dependency

7.6 Potential Deployment

- Defense organizations: sensitive documents, credentials, software entitlements and controlled transfers.
- Government departments: cross-department digital assets and authorization workflows.
- Critical infrastructure: controlled access to operational records and licenses.
- Enterprise environments: contracts, certificates, intellectual property and compliance records.
- Long-term expansion: department → ministry → inter-agency → national-scale controlled federation.

8. CHALLENGES AND LIMITATIONS

8.1 Key Management

Passwordless cryptography shifts responsibility from passwords to key protection. Loss or compromise of a private key can disrupt access or create security risk. A production system therefore needs secure key storage, backup/recovery, revocation, rotation, and organizational identity verification.

8.2 Performance and Latency

Signature verification, policy evaluation, asset governance, and validator synchronization introduce processing steps. At high concurrency, the backend and validator layer may become bottlenecks. Caching, asynchronous processing, optimized policy evaluation, and horizontal scaling should be evaluated during performance testing.

8.3 Insider and Privilege Risks

RBAC reduces broad privilege but does not eliminate misuse by authorized users. Hybrid ABAC, multi-party approval, detailed audit records, and revocation mechanisms reduce the blast radius of inappropriate actions.

8.4 Node Availability

A permissioned network depends on the availability of its validator nodes. Node outages, network partitions, and configuration mismatches can affect synchronization. Production deployments should define backup nodes, monitoring, recovery procedures, and a formally specified consensus/fault model.

8.5 Integration Challenges

Government and defense environments often contain legacy applications with different identity models, data formats, and network boundaries. API gateways, adapters, staged migration, and clearly defined trust boundaries are needed for integration.

8.6 Current Prototype Limitations

- The prototype is not a certified production blockchain or classified-information system.
- Actual consensus selection and validator hardening require deployment-specific engineering.
- Enterprise NFT semantics must be adapted to the organization's legal ownership model.
- Security claims require independent penetration testing, code review, and formal threat modeling before production use.
- Compliance with applicable laws and policies must be validated by the deploying organization.

9. IMPACT AND BENEFITS

9.1 Security Impact

TrustChain Pro aims to reduce unauthorized access by requiring cryptographically verifiable identity and layered authorization. The combination of RBAC and ABAC means that possession of valid credentials is not intended to be sufficient for every sensitive operation.

9.2 Operational Benefits

- Passwordless authentication can reduce dependence on reusable credentials.
- Automated policy evaluation reduces manual authorization steps.
- Real-time notifications improve visibility into critical asset events.
- Structured workflows make approval and transfer processes easier to trace.

9.3 Asset Governance Benefits

- Enterprise NFT representations provide unique digital asset identifiers.
- Ownership changes can be recorded as governed transactions.
- Chaincode rules can enforce who may mint, allocate, or transfer assets.
- Critical transfers can require multiple authorized parties.

9.4 Audit and Accountability

A tamper-evident transaction history provides stronger evidence than an ordinary editable record. Auditors can verify cryptographic links, inspect authorization decisions, and trace asset movements across the platform.

9.5 Government and Defense Use Cases

Use Case	Potential Application
Sensitive Documents	Controlled access and traceable ownership
Credentials & Certificates	Issue, allocate and verify organizational credentials
Software Licenses	Track authorized ownership and transfer
Procurement Assets	Record assignment and controlled transfer
Inter-Department Sharing	Policy-based access across organizational boundaries
Audit & Investigation	Searchable, hash-verifiable event history

IDENTITY → ACCESS → GOVERNANCE → OWNERSHIP → AUDIT

10. FUTURE ENHANCEMENTS

10.1 Advanced Key Recovery

Integrate hardware-backed key storage, organizational recovery ceremonies, multi-party recovery, and secure key rotation so that identity continuity can be maintained without weakening the trust model.

10.2 Hardware-Backed Identity

Future deployments can evaluate smart cards, secure elements, platform authenticators, or hardware security modules for stronger protection of private keys.

10.3 Advanced Policy Engine

The ABAC engine can be extended with device posture, geofencing where permitted, threat-intelligence signals, risk history, asset classification, and temporary emergency access policies.

10.4 Cross-Department Federation

A future federation model could allow multiple departments to maintain independent policy domains while sharing verified identity and asset events through controlled trust relationships.

10.5 Advanced SIEM Integration

Security events can be forwarded to enterprise SIEM systems for correlation with endpoint, network, authentication, and incident-response telemetry.

10.6 Mobile and Offline Workflows

Secure mobile clients and controlled offline workflows could support field personnel while preserving local cryptographic verification and later synchronization. Offline operation would require careful replay protection and conflict-resolution policies.

11. CONCLUSION

TrustChain Pro presents a unified approach to secure identity, contextual access control, digital asset ownership, governance, and auditability for defense and government-oriented environments. The project combines established web technologies with cryptographic identity and permissioned ledger concepts so that the solution remains practical rather than dependent on public cryptocurrency infrastructure.

The central design principle is layered trust: a user must possess a valid cryptographic identity, have an appropriate organizational role, satisfy the required dynamic security context, and operate within the asset's governance policy. The resulting action is then recorded in a tamper-evident history. This connects identity, authorization, ownership, and accountability in a single workflow.

The prototype architecture is technically feasible and can be extended toward real deployments, but production use would require organization-specific threat modeling, performance validation, key-management controls, infrastructure hardening, independent security assessment, and legal/compliance review. With these controls, the proposed architecture provides a structured foundation for secure digital asset management across controlled organizational environments.

12. REFERENCES

- [W3C — Decentralized Identifiers \(DID Core\)](#)
- [NIST — SP 800-207: Zero Trust Architecture](#)
- [Government of India — Digital Personal Data Protection Act, 2023](#)
- [W3C — Web Cryptography API](#)
- [Node.js — Official Documentation](#)
- [Express.js — Official Documentation](#)
- [MongoDB — Official Documentation](#)
- [Socket.IO — Official Documentation](#)
- [Smart India Hackathon — Official Website](#)
- [Bharat Electronics Limited — Official Website](#)

PROJECT DEMONSTRATION LINKS

GitHub Repository — [GitHub - TechForce-coder/Trustchain-Pro · GitHub](#)

Animated Demo — [TrustChain Pro: Zero-Trust Defense Asset Management & Threat Isolation Demo](#)

Prototype Demonstration Video — [TrustChain Pro: Live Prototype & Working Model Demonstration | SIH 2026](#)

APPENDIX A – CORE WORKFLOW

Step	Event	Security Decision
1	User request	Identify actor and requested resource
2	Cryptographic sign	Verify ECDSA signature
3	RBAC check	Verify role and baseline permission
4	ABAC check	Evaluate current context
5	Governance check	Verify asset lifecycle rule
6	Commit	Create governed ledger event
7	Broadcast	Synchronize authorized nodes/clients
8	Audit	Retain and verify evidence

APPENDIX B – CORE PROJECT TERMINOLOGY

Term	Meaning
DID	Decentralized Identifier used to represent an identity
ECDSA P-256	Elliptic-curve digital signature algorithm using the P-256 curve
RBAC	Role-Based Access Control
ABAC	Attribute-Based Access Control
Enterprise NFT	Unique tokenized representation of an organizational digital asset
Chaincode	Programmable governance logic for controlled ledger operations
Permissioned Ledger	Ledger whose participating validators are controlled by an organization
Hash Chain	Sequence of records linked through cryptographic hashes
SIEM	Security Information and Event Management

APPENDIX C – JUDGE-FOCUSED VALUE PROPOSITION

- Secure Identity: Passwordless DID + ECDSA P-256.
- Layered Authorization: RBAC + dynamic ABAC.
- Digital Ownership: Enterprise NFTs with traceable ownership.
- Automated Governance: Chaincode-controlled mint, allocation and transfer.
- Trustworthy Evidence: Hash-linked tamper-evident audit history.
- Deployment Practicality: MERN, APIs, WebSockets and permissioned infrastructure.
- Expansion Path: Department → Ministry → Inter-Agency → National controlled deployment.

PROJECT LINKS

TrustChain Pro • TECH FORCE • ZIH061

DEMONSTRATION & DEVELOPMENT

GitHub Repository — [GitHub - TechForce-coder/Trustchain-Pro · GitHub](#)

Animated Demo — [TrustChain Pro: Zero-Trust Defense Asset Management & Threat Isolation Demo](#)

Prototype Demonstration Video — [TrustChain Pro: Live Prototype & Working Model Demonstration | SIH 2026](#)

OFFICIAL TECHNICAL REFERENCES

- [W3C — Decentralized Identifiers \(DID Core\)](#)
- [NIST — SP 800-207: Zero Trust Architecture](#)
- [W3C — Web Cryptography API](#)
- [Node.js — Official Documentation](#)
- [MongoDB — Official Documentation](#)
- [Smart India Hackathon — Official Website](#)